

Auth & Onboarding PRD

Addendum v1.1

March 28, 2026 | VaNiBase Framework + KI-Prime

Change Summary

This addendum documents decisions made during the March 28, 2026 mentor session that refine or extend the Auth & Onboarding PRD v1.0. Claude Code sessions should read this addendum alongside the original PRD.

#	Change	Impact	PRD Section
1	VN_error_log table added	New migration (006), error handling middleware, error logger service	5 (Schema)
2	Onboarding: only mandatory steps tracked in DB	VN_tenant_onboarding seeded only for mandatory steps; non-mandatory use direct CRUD	6 (Onboarding)
3	onboarding_complete field in /auth/me	Replaces X-Onboarding-Incomplete header approach; AuthContext stores this field	10 (API)
4	Theme resolution is client-side only	No server-side theme middleware; ThemeProvider in shell resolves chain from AuthContext	7 (Theme)
5	Forgot password promoted to P1	Endpoint + reset page built in MVP; email/WhatsApp delivery deferred	10 (API)

1. New Table: VN_error_log

Add to PRD Section 5 (Database Schema) as Section 5.4.

1.1 Purpose

Captures server-side errors for production debugging. In development (NODE_ENV=development), errors are logged to console only. In production, errors are persisted to VN_error_log for post-incident analysis.

1.2 Schema

Column	Type	Nullable	Notes
id	UUID	No	PK, gen_random_uuid()
tenant_id	UUID	Yes	FK → VN_tenants.id
user_id	UUID	Yes	FK → VN_users.id
error_code	VARCHAR(50)	No	App-defined code (e.g. AUTH_FAILED)
message	TEXT	No	Human-readable error message
stack	TEXT	Yes	Stack trace (production only)
endpoint	VARCHAR(255)	Yes	Request path (e.g. /api/v1/auth/login)
method	VARCHAR(10)	Yes	HTTP method (GET, POST, etc.)
severity	VARCHAR(20)	No	DEFAULT 'error'. Values: error, warning, critical
metadata	JSONB	No	DEFAULT '{}'. Request body, headers, etc.
created_at	TIMESTAMPTZ	No	DEFAULT now()

1.3 Migration

File: 006_vn_error_log.sql

Depends on: 001 (VN_tenants), 002 (VN_users)

Index strategy (recommended, Claude Code may add to RECOMMENDATIONS.md):

```
CREATE INDEX idx_error_log_tenant_created ON VN_error_log (tenant_id, created_at DESC);  
CREATE INDEX idx_error_log_severity ON VN_error_log (severity, created_at DESC);
```

1.4 Error Handling Architecture

Components to Build

Component	Location	Responsibility
error-handler.ts	framework/middleware/	Express error interceptor. Last middleware in chain. Returns { error: { code, message, details? }, status }.
error-logger.ts	framework/services/	Writes to console (dev) or VN_error_log (prod). Extracts tenant_id, user_id from ctx.
error-boundary.tsx	shell/src/components/	React ErrorBoundary. Catches render errors, shows fallback UI with retry button.

Error Response Format

```
{ "error": { "code": "VALIDATION_ERROR", "message": "Email is required", "details": { "field": "email" } }, "status": 400 }
```

Severity Levels

error — Unhandled exceptions, failed DB operations, auth failures

warning — Rate limit approaches, deprecated endpoint usage

critical — DB connection loss, JWT secret mismatch, migration failures

2. Onboarding: Mandatory-Only DB Tracking

Replaces PRD Section 6 onboarding tracking logic. The wizard UI and step definitions in ShellConfig remain unchanged.

2.1 Revised Rule

Only mandatory steps are seeded as rows in VN_tenant_onboarding. Non-mandatory steps have no DB tracking rows. The wizard UI still presents all steps (from ShellConfig.onboarding.steps[]), but non-mandatory steps save data via their direct CRUD endpoint and are not tracked for completion.

2.2 ShellConfig Definition (Product-Specific)

```
// shell.config.ts (KI-Prime example)
onboarding: {
  steps: [
    { id: 'user_profile',      order: 1, mandatory: true,  component:
'OnboardUserProfile' },
    { id: 'business_profile',  order: 2, mandatory: true,  component:
'OnboardBusiness' },
    { id: 'theme_selection',   order: 3, mandatory: false, component:
'OnboardTheme' },
    { id: 'invite_team',       order: 4, mandatory: false, component:
'OnboardInvite' },
    { id: 'risk_preferences',  order: 5, mandatory: false, component:
'OnboardPreferences' },
    { id: 'import_data',       order: 6, mandatory: false, component:
'OnboardImport' },
  ]
}
```

2.3 Seeding on Registration

When POST /api/v1/auth/register succeeds and a new tenant is created:

1. Read ShellConfig.onboarding.steps[] and filter where mandatory === true.
2. INSERT one row per mandatory step into VN_tenant_onboarding with status = 'pending', completed_at = NULL.
3. Non-mandatory steps are NOT seeded. No rows exist for them.

2.4 Completion Logic

GET /api/v1/onboarding/status

Query: SELECT * FROM VN_tenant_onboarding WHERE tenant_id = ?

Compute: complete = (all rows have status = 'completed')

Return: { complete: boolean, steps: [{ step_id, status, completed_at }], next_incomplete_step: string | null }

Note: 'steps' array only contains mandatory steps (the ones with DB rows). Non-mandatory steps are not included.

PATCH /api/v1/onboarding/step

Accepts: { step_id: string, status: 'completed', metadata?: object }

Validates: step_id must exist in VN_tenant_onboarding for this tenant (i.e., must be a mandatory step).

If step_id is not found in DB → 400 (not a tracked step).

Updates: status = 'completed', completed_at = now(), metadata = provided JSONB.

Non-Mandatory Step Data Flow

When the wizard UI renders a non-mandatory step (e.g., theme_selection), the component calls its own CRUD endpoint directly:

- theme_selection → PATCH /api/v1/tenant/profile { theme_id }
- invite_team → POST /api/v1/auth/invite
- risk_preferences → PATCH /api/v1/tenant/preferences
- import_data → POST /api/v1/skills/import

The wizard advances to the next step regardless of whether the user interacts with the non-mandatory step or clicks Skip/Next.

2.5 Wizard Navigation

Step	Mandatory	Skip Button	On Complete/Skip
user_profile	Yes	No	PATCH /onboarding/step → next
business_profile	Yes	No	PATCH /onboarding/step → next
theme_selection	No	Yes (or Next)	Direct CRUD → next
invite_team	No	Yes (or Next)	Direct CRUD → next
risk_preferences	No	Yes (or Next)	Direct CRUD → next
import_data	No	Yes (Finish)	Direct CRUD → redirect to dashboard

3. /auth/me Response: onboarding_complete Field

Replaces the X-Onboarding-Incomplete header approach from PRD v1.0 Section 10.

3.1 Updated /auth/me Response

```
{
  "id": "uuid",
  "email": "user@example.com",
  "full_name": "Charan Kamal",
  "phone": "+91-9876543210",
  "role_id": "owner",
  "avatar_url": "https://firebase...",
  "tenant": {
    "id": "uuid",
    "name": "Vikuna Technologies",
    "theme_id": "vikuna-black",
    "logo_url": "https://firebase...",
    "onboarding_complete": true
  },
  "preferences": { "theme": "classic-elegant" }
}
```

3.2 How onboarding_complete is Computed

In the /auth/me handler, after fetching user + tenant data:

```
SELECT count(*) as pending FROM VN_tenant_onboarding
WHERE tenant_id = $1 AND status != 'completed';
```

If pending === 0 → onboarding_complete = true. Otherwise false.

3.3 Client-Side Handling (AuthContext)

AuthContext stores the full /auth/me response including tenant.onboarding_complete.

Redirect logic in the shell's root layout or auth guard:

1. If onboarding_complete === false AND user.role_id === 'owner' → redirect to /onboarding
2. If onboarding_complete === false AND user.role_id !== 'owner' → show pending block screen
3. If onboarding_complete === true → proceed to dashboard

On login and token refresh, /auth/me is called and AuthContext is updated.

3.4 No Custom Headers Required

The X-Onboarding-Incomplete and X-Onboarding-Blocked headers from PRD v1.0 are removed. The onboarding_complete boolean in the /auth/me response replaces both. Auth middleware does NOT need to check onboarding status on every request — the client handles redirection based on AuthContext state.

4. Theme Resolution: Client-Side Only

Clarifies PRD Section 7 (Theme Resolution). No server-side theme middleware is needed.

4.1 Resolution Chain

The shell's ThemeProvider component resolves the active theme using this priority:

1. `user.preferences.theme` (user-level preference from `/auth/me`)
2. `tenant.theme_id` (tenant-level config from `/auth/me`)
3. `ShellConfig.theme.default` (product default in `shell.config.ts`, e.g., 'vikuna-black')
4. 'classic-elegant' (framework hardcoded fallback)

4.2 Data Source

Both `user.preferences.theme` and `tenant.theme_id` are returned in the `/auth/me` response. The ThemeProvider reads them from AuthContext. No additional API call is needed.

4.3 Pre-Login Pages

For pages rendered before authentication (register, forgot-password, invite-accept):

Use `ShellConfig.theme.default` directly. No AuthContext is available at this point.

4.4 Server-Side Impact

None. The API stores theme values (PATCH `/auth/preferences`, PATCH `/tenant/profile`) and returns them in `/auth/me`. It does not resolve themes or set theme-related headers.

5. Forgot Password: Promoted to P1

Updates PRD Section 10 (API Summary) priority for the forgot-password endpoint.

5.1 Updated Endpoint

Endpoint	Method	Priority	Notes
<code>/api/v1/auth/forgot-password</code>	POST	P1	Token returned in API response. Email/WhatsApp delivery is future scope.
<code>/api/v1/auth/reset-password</code>	POST	P1	Accepts token + new password. Validates token from VN_password_resets, updates password with bcrypt 12.

5.2 Reset Password Flow (MVP)

1. User submits email to POST `/api/v1/auth/forgot-password`
2. API generates a random token, hashes with SHA-256, stores in VN_password_resets (token_hash, user_id, expires_at = now + 1 hour)
3. API returns { token: '<raw_token>' } in the response body (NOT emailed)
4. User navigates to reset-password page, enters token + new password
5. POST `/api/v1/auth/reset-password` validates token, updates password, deletes the reset record

Future: Step 3 will dispatch the token via email/WhatsApp instead of returning it in the API response.

6. Updated Claude Code Prompts

These prompts supersede the ones in the handover document where they conflict. Changes are marked with [ADDENDUM].

6.1 Prompt 3 (Revised): VaNiBase Onboarding Framework

Read CLAUDE.md, Auth-Onboarding-PRD-v1.docx (Section 6), and Auth-Onboarding-PRD-Addendum-v1.1.docx (Section 2).

Build the onboarding framework:

1. GET /api/v1/onboarding/status
 - Query VN_tenant_onboarding for this tenant
 - Return { complete: boolean, steps: [{step_id, status, completed_at}], next_incomplete_step }
 - complete = all rows have status 'completed'
2. PATCH /api/v1/onboarding/step
 - Accepts { step_id, status: 'completed', metadata? }
 - Validate step_id exists in VN_tenant_onboarding for this tenant
 - If not found -> 400 (not a tracked step)
 - Update status, completed_at, metadata
3. On registration (in register handler):
 - Import ShellConfig.onboarding.steps[]
 - Filter mandatory === true
 - INSERT rows into VN_tenant_onboarding with status='pending'
 - Do NOT seed non-mandatory steps
4. Update GET /api/v1/auth/me:
 - Add onboarding_complete to tenant object in response
 - Compute: SELECT count(*) FROM VN_tenant_onboarding WHERE tenant_id=\$1 AND status!='completed'
 - If count=0 -> true, else false
5. Do NOT add X-Onboarding-Incomplete or X-Onboarding-Blocked headers.
Do NOT modify auth middleware to check onboarding status.
Client handles redirection via AuthContext.
6. Shell route: (onboarding)/page.tsx
 - Reads all steps from ShellConfig (mandatory + non-mandatory)
 - Renders wizard, navigates steps in order
 - Mandatory steps: call PATCH /onboarding/step on complete
 - Non-mandatory steps: call direct CRUD endpoint, then advance
 - Skip button on non-mandatory steps

```
- Pending block screen for invited users (role != owner +  
onboarding_complete=false)
```

Follow Addendum v1.1 Section 2 exactly. Write recommendations to RECOMMENDATIONS.md.

6.2 Additional Prompt: Reset Password Endpoint

Read CLAUDE.md and Auth-Onboarding-PRD-Addendum-v1.1.docx (Section 5).

Build these endpoints in framework/routes/auth.ts:

1. POST /api/v1/auth/forgot-password
 - Accepts { email }
 - Look up user by email. If not found, return 200 (no user enumeration).
 - Generate crypto.randomBytes(32) token
 - Hash with SHA-256, store in VN_password_resets (token_hash, user_id, expires_at = now + 1hr)
 - Return { token: '<raw_token>' } in response (MVP only, email dispatch future)
2. POST /api/v1/auth/reset-password
 - Accepts { token, new_password }
 - Hash token with SHA-256, look up in VN_password_resets
 - Validate: exists, not expired, not used
 - Update user password with bcrypt 12
 - Delete the reset record (or mark used)
 - Return 200 { message: 'Password reset successful' }

Follow the PRD exactly. Write recommendations to RECOMMENDATIONS.md.

Appendix: Revised API Priority Table

Complete endpoint list with updated priorities from this addendum.

Endpoint	Method	Priority	Status / Notes
/api/v1/auth/register	POST	P1	Already built
/api/v1/auth/login	POST	P1	Already built
/api/v1/auth/refresh	POST	P1	Already built
/api/v1/auth/logout	POST	P1	Already built
/api/v1/auth/me	GET	P1	Already built. ADD onboarding_complete to tenant object [ADDENDUM]
/api/v1/auth/sessions/revoke	DELETE	P1	Already built
/api/v1/auth/preferences	PATCH	P1	Already built
/api/v1/auth/invite	POST	P1	To build
/api/v1/auth/invite/accept	POST	P1	To build
/api/v1/auth/invitations	GET	P1	To build
/api/v1/auth/invitations/:id	DELETE	P2	To build
/api/v1/auth/change-password	POST	P1	To build
/api/v1/auth/forgot-password	POST	P1	To build [ADDENDUM: promoted from P2]
/api/v1/auth/reset-password	POST	P1	To build [ADDENDUM: new endpoint]
/api/v1/onboarding/status	GET	P1	To build
/api/v1/onboarding/step	PATCH	P1	To build [ADDENDUM: mandatory steps only]
/api/v1/tenant/profile	PATCH	P1	To build